Université de Yaoundé I
Faculté des Sciences – Département d'Informatique
INF4067 / INF461 – Principes et processus du logiciel

RAPPORT D'ÉTUDE

Patron de construction Factory (Fabrique)

Devoir n° 1 – Design Patterns : patrons de construction

Étudiant : GOYE AWOUMA CHAINE LEONEL

Enseignant : Pr KIMBI XAVERIA

Dépôt GitHub :
https://github.com/LeonelAwouma/PatronDeConstruction_Factory_GOYE_AWOUMA_CHAINE_LEONEL

Date : 10 novembre 2026

1. Introduction

Ce rapport présente l'étude du patron de construction **Factory** (Fabrique), réalisée dans le cadre du devoir n° 1 du cours INF4067 / INF461. Il suit le plan demandé : nom du patron, modèle générique, exemple du cours, exemple modifié, code source et lien vers le dépôt GitHub où le code est stocké.

Les patrons de création ont pour vocation d'abstraire les mécanismes de création d'objets. Un système qui les utilise devient indépendant de la façon dont les objets sont créés et, en particulier, des mécanismes d'instanciation des classes concrètes. Ils encapsulent l'utilisation des classes concrètes et favorisent ainsi l'utilisation des interfaces.

2. Nom du patron

Factory (en français **Fabrique**). C'est un patron de construction : il permet de créer un objet dont le type dépend du contexte, cet objet faisant partie d'un ensemble de sous-classes. L'objet retourné est toujours du type de la classe mère et le polymorphisme permet de spécialiser les traitements. Le support de cours le présente sous deux formes :

- **Factory Method** : une fabrique possède une méthode de création qui reçoit les données nécessaires (par exemple une chaîne « A », « B », « C ») pour déterminer le type de l'objet à créer.
- **Factory avec fabrique abstraite** : une classe de fabrique abstraite déclare la méthode de fabrique, et ce sont ses sous-classes concrètes qui créent les objets.

2.1 Problème résolu

Dans la plupart des langages objet, la création d'objets se fait avec l'opérateur `new`. Son utilisation est « mauvaise » car elle crée un **couplage fort** entre l'appelant et la classe instanciée : une relation classe-à-classe est fragile. De plus, la classe passée à `new` ne peut pas être remplacée par une variable, il est donc difficile de paramétrer la création. Enfin, lorsque le client utilise une instruction conditionnelle pour choisir la classe, chaque changement dans la hiérarchie des classes à instancier demande des modifications dans le code du client.

La solution est une relation **classe-à-interface** : le client ne connaît que la classe mère abstraite et délègue la création à une fabrique. Le couplage devient faible et le polymorphisme spécialise les traitements.

2.2 Conditions d'utilisation

- Le client ne peut déterminer le type d'objet à créer qu'à l'**exécution**.
- Il y a une volonté de **centraliser** la création d'objets.
- L'objet à créer fait partie d'un ensemble de sous-classes et l'objet retourné est toujours du type de la classe mère.

3. Modèle générique

3.1 Forme 1 : Factory Method

Structure générique (support, slide 17) : le **Client** utilise une **Fabrique** dont la méthode `methodeFabrique() : Produit` instancie un **ProduitConcret**, sous-classe de **Produit**.

Participant	Rôle
Produit	Classe abstraite (ou interface) dont les sous-classes seront instanciées.
ProduitConcret	Les sous-classes concrètes à instancier.
Fabrique	Contient la méthode de fabrique ; elle reçoit les données nécessaires pour choisir la classe à instancier.
Client	Programme qui fait appel à la fabrique pour obtenir des objets de type Produit.

3.2 Forme 2 : Factory avec fabrique abstraite

Structure générique (support, slide 24) : une **FabriqueAbstraite** abrite la signature de la méthode de fabrique et l'implémentation des méthodes qui l'invoquent ; une ou plusieurs **FabriqueConcrete** implémentent la méthode de fabrique et instancient le **ProduitConcret**.

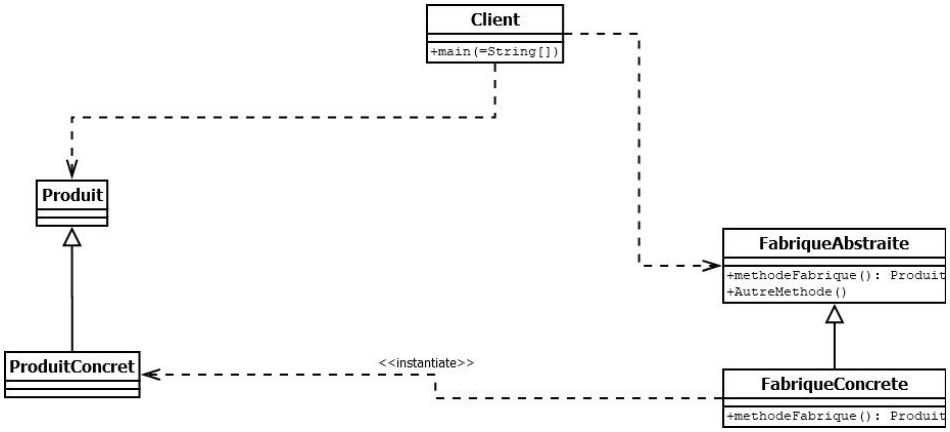


Figure 1 – Modèle générique de la forme 2, reproduit avec DIA d'après la slide 24.

Participant	Rôle
FabriqueAbstraite	Classe abstraite : signature de la méthode de fabrique et méthodes qui l'invoquent.
FabriqueConcrete	Classe concrète qui implémente la méthode de fabrique. Il peut y en avoir plusieurs.
Produit / ProduitConcret	Classe abstraite et ses sous-classes concrètes à instancier.
Client	Programme qui fait appel à la fabrique.

3.3 Différence avec Abstract Factory

Factory crée **un** produit d'une hiérarchie. Abstract Factory fournit une interface unique pour créer **une famille** d'objets liés (par exemple Scooter et Automobile, en version Essence ou Électrique) sans avoir à connaître les classes à instancier. Abstract Factory est un patron distinct, traité dans un autre dépôt.

4. Exemple du cours

Énoncé : on gère trois types de produits dans un système. Le programme qui manipule ces produits ne connaît le type de produit à exécuter que lors de l'exécution. On utilise le patron Factory pour gérer cette situation, d'abord avec Factory Method (slides 18 à 23), puis avec une fabrique abstraite (slides 26 à 30).

4.1 Modélisation – Factory Method

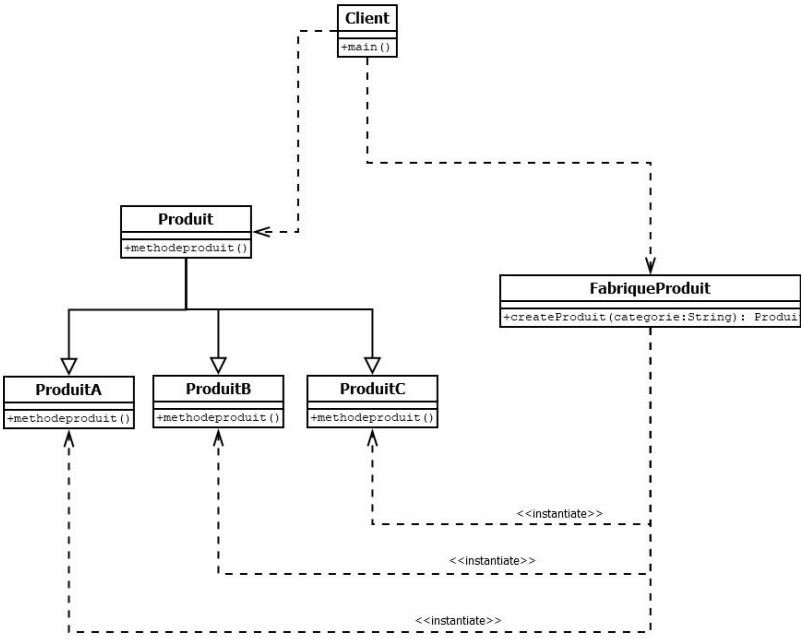


Figure 2 – Diagramme de classes de l'exemple du cours, forme Factory Method (DIA, d'après la slide 19).

Remarque : le diagramme de la slide 19 présente `Produit` comme une interface alors que le code des slides 20 et 28 la déclare `abstract class`. Le rendu suit le code : `Produit` est une classe abstraite.

4.2 Modélisation – fabrique abstraite

La hiérarchie des fabriques (slide 27) : `ProduitFactory` abstraite avec `getProduit()` qui appelle `createProduit()` abstraite ; `ProduitFactoryA`, `ProduitFactoryB` et `ProduitFactoryC` instancient respectivement `ProduitA`, `ProduitB` et `ProduitC`. Le modèle générique correspondant est la figure 1.

4.3 Code source – Factory Method

Cours/src/factorymethod/Produit.java

```
package factorymethod;

/**
 * Produit : classe abstraite dont les sous-classes seront instanciees par la fabrique.
 * L'objet retourne par la fabrique est toujours de ce type (classe mere).
 */
public abstract class Produit {
    public abstract void methodeproduit();
}
```

Cours/src/factorymethod/ProduitA.java

```
package factorymethod;

public class ProduitA extends Produit {
    public ProduitA() {
    }

    @Override
    public void methodeproduit() {
        System.out.println("je suis un produit de type A");
        System.out.println("produitA.methodeproduit()");
    }
}
```

Cours/src/factorymethod/ProduitB.java

```
package factorymethod;

public class ProduitB extends Produit {
    @Override
    public void methodeproduit() {
        System.out.println("je suis un produit de type B");
        System.out.println("produitB.methodeproduit()");
    }
}
```

Cours/src/factorymethod/ProduitC.java

```
package factorymethod;

public class ProduitC extends Produit {
    @Override
    public void methodeproduit() {
        System.out.println("je suis un produit de type C");
        System.out.println("produitC.methodeproduit()");
    }
}
```

Cours/src/factorymethod/FabriqueProduit.java

```
package factorymethod;

/**
 * FabriqueProduit : contient la methode de fabrique createProduit(type).
 * Elle recoit la donnee necessaire (le type) pour determiner quel produit concret instancier.
 * Le client ne manipule jamais directement ProduitA, ProduitB ou ProduitC.
 */
public class FabriqueProduit {
    public static final String typeA = "A";
    public static final String typeB = "B";
    public static final String typeC = "C";

    public Produit createProduit(String type) {
        Produit produit = null;
        switch (type) {
            case typeA:
                produit = new ProduitA();
                break;
            case typeB:
                produit = new ProduitB();
                break;
            case typeC:
```

```
                produit = new ProduitC();
                break;
            default:
                throw new IllegalArgumentException("Type de produit inconnu");
        }
        return produit;
    }
}
```

Cours/src/factorymethod/Client.java

```
package factorymethod;

/**
 * Client : programme qui fait appel a la fabrique pour obtenir des objets de type Produit.
 * Le type a creer n'est connu qu'a l'execution.
 */
public class Client {
    public static void main(String[] args) {
        System.out.println(".....Methode 1: Factory Method.....");

        FabriqueProduit fabriqueproduit = new FabriqueProduit();
        Produit produit = null;

        // affiche un produit de type A
        produit = fabriqueproduit.createProduit(FabriqueProduit.typeA);
        produit.methodeproduit();
        // affiche un produit de type B
        produit = fabriqueproduit.createProduit(FabriqueProduit.typeB);
        produit.methodeproduit();
        // affiche un produit de type C
        produit = fabriqueproduit.createProduit(FabriqueProduit.typeC);
        produit.methodeproduit();

        // produit inexistant : la fabrique leve une exception
        String d = "D";
        try {
            produit = fabriqueproduit.createProduit(d);
            produit.methodeproduit();
        } catch (IllegalArgumentException e) {
            System.out.println("Erreur : " + e.getMessage() + " (" + d + ")");
        }
    }
}
```

4.4 Code source – fabrique abstraite

Cours/src/factoryabstraite/Produit.java

```
package factoryabstraite;

public abstract class Produit {
    public abstract void methodeproduit();
}
```

Cours/src/factoryabstraite/ProduitA.java

```
package factoryabstraite;

public class ProduitA extends Produit {
    @Override
    public void methodeproduit() {
        System.out.println("je suis un produit de type A");
        System.out.println("produitA.methodeproduit()");
    }
}
```

Cours/src/factoryabstraite/ProduitB.java

```
package factoryabstraite;

public class ProduitB extends Produit {
    @Override
    public void methodeproduit() {
        System.out.println("je suis un produit de type B");
        System.out.println("produitB.methodeproduit()");
```

```
    }
}
```

Cours/src/factoryabstraite/ProduitC.java
```
package factoryabstraite;

public class ProduitC extends Produit {
    @Override
    public void methodeproduit() {
        System.out.println("je suis un produit de type C");
        System.out.println("produitC.methodeproduit()");
    }
}
```

Cours/src/factoryabstraite/ProduitFactory.java
```
package factoryabstraite;

/**
 * FabriqueAbstraite : abrite la signature de la methode de fabrique (createProduit)
 * et l'implementation des methodes qui l'invoquent (getProduit).
 * Ce sont les sous-classes concretes qui decident quel produit instancier.
 */
public abstract class ProduitFactory {
    public Produit getProduit() {
        return createProduit();
    }

    protected abstract Produit createProduit();
}
```

Cours/src/factoryabstraite/ProduitFactoryA.java
```
package factoryabstraite;

/**
 * FabriqueConcrete : implemente la methode fabrique pour creer un ProduitA.
 */
public class ProduitFactoryA extends ProduitFactory {
    @Override
    protected Produit createProduit() {
        Produit produit = null;
        produit = new ProduitA();
        return produit;
    }
}
```

Cours/src/factoryabstraite/ProduitFactoryB.java
```
package factoryabstraite;

/**
 * FabriqueConcrete : implemente la methode fabrique pour creer un ProduitB.
 */
public class ProduitFactoryB extends ProduitFactory {
    @Override
    protected Produit createProduit() {
        Produit produit = null;
        produit = new ProduitB();
        return produit;
    }
}
```

Cours/src/factoryabstraite/ProduitFactoryC.java
```
package factoryabstraite;

/**
 * FabriqueConcrete : implemente la methode fabrique pour creer un ProduitC.
 */
public class ProduitFactoryC extends ProduitFactory {
    @Override
    protected Produit createProduit() {
        Produit produit = null;
        produit = new ProduitC();
        return produit;
    }
```

```
}
```

Cours/src/factoryabstraite/Client.java

```
package factoryabstraite;


public class Client {
    public static void main(String[] args) {
        System.out.println(".....Methode 2: Factory avec fabrique
abstraite.....");
        ProduitFactory produitfactoryA = new ProduitFactoryA();
        ProduitFactory produitfactoryB = new ProduitFactoryB();
        ProduitFactory produitfactoryC = new ProduitFactoryC();
        Produit produit = null;
        // affiche un produit de type A
        produit = produitfactoryA.getProduit();
        produit.methodeproduit();
        // affiche un produit de type B
        produit = produitfactoryB.getProduit();
        produit.methodeproduit();
        // affiche un produit de type C
        produit = produitfactoryC.getProduit();
        produit.methodeproduit();
    }
}
```

4.5 Résultat de l'exécution

```
.....Methode 1: Factory Method.....
je suis un produit de type A
produitA.methodeproduit()
je suis un produit de type B
produitB.methodeproduit()
je suis un produit de type C
produitC.methodeproduit()
Erreur : Type de produit inconnu (D)


.....Methode 2: Factory avec fabrique abstraite.....
je suis un produit de type A
produitA.methodeproduit()
je suis un produit de type B
produitB.methodeproduit()
je suis un produit de type C
produitC.methodeproduit()
```

5. Exemple modifié : système de notifications

Énoncé : une application doit prévenir un utilisateur par **e-mail**, **SMS** ou **notification push**. Le canal n'est connu qu'à l'exécution : il dépend de la préférence de l'utilisateur ou d'un argument de la ligne de commande. Le programme ne doit manipuler que la classe mère `Notification`. On utilise le patron Factory dans ses deux formes.

Participant du modèle	Exemple du cours	Exemple modifié
Produit	Produit, methodeproduit()	Notification, envoyer(destinataire, message)
ProduitConcret	ProduitA, ProduitB, ProduitC	EmailNotification, SMSNotification, PushNotification
Fabrique (forme 1)	FabriqueProduit.createProduit(type)	NotificationFactory.creerNotification(canal)

Participant du modèle	Exemple du cours	Exemple modifié
FabriqueAbstraite (forme 2)	ProduitFactory : getProduit(), createProduit()	NotificationFactory : notifier(), creerNotification()
FabriqueConcrete (forme 2)	ProduitFactoryA, B, C	EmailNotificationFactory, SMSNotificationFactory, PushNotificationFactory
Client	Client.main	Client.main (canal passé en argument)

5.1 Modélisation – Factory Method

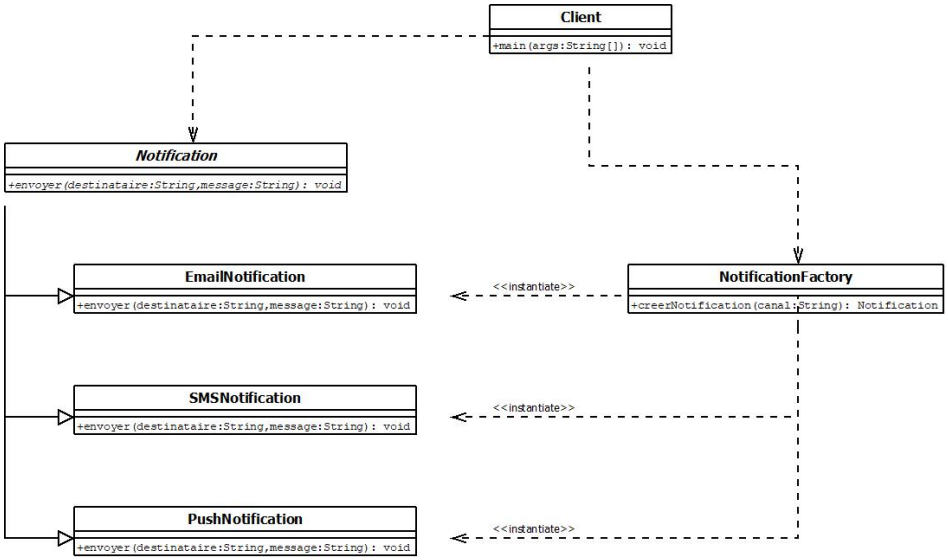


Figure 3 – Variante Notifications, forme Factory Method (DIA).

5.2 Modélisation – fabrique abstraite

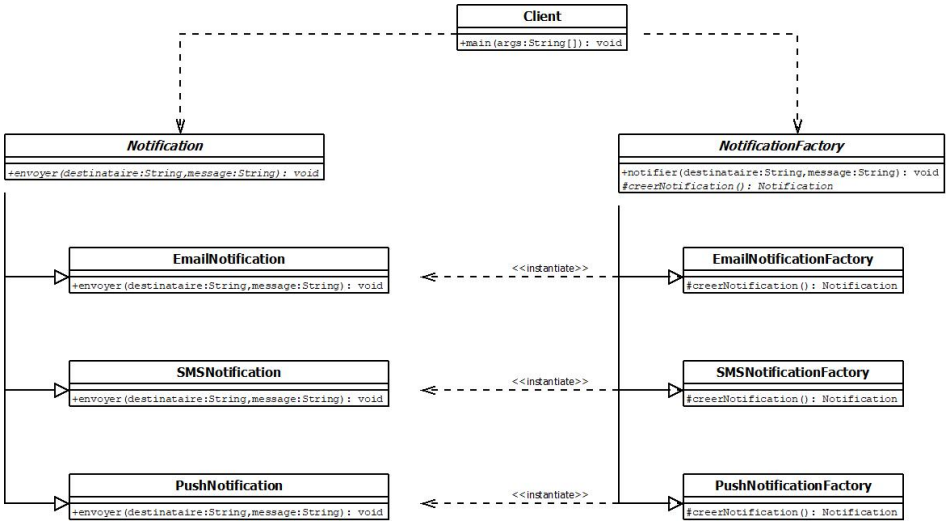


Figure 4 – Variante Notifications, forme fabrique abstraite (DIA).

5.3 Code source – Factory Method

variantes/src/factorymethod/Notification.java

```
package factorymethod;

/**
 * Notification : classe abstraite (role Produit).
 * Toute notification sait s'envoyer a un destinataire avec un message.
 */
public abstract class Notification {
    public abstract void envoyer(String destinataire, String message);
}
```

variantes/src/factorymethod/EmailNotification.java

```
package factorymethod;

public class EmailNotification extends Notification {
    @Override
    public void envoyer(String destinataire, String message) {
        System.out.println("[EMAIL] A : " + destinataire);
        System.out.println("        Objet : Notification");
        System.out.println("        Corps : " + message);
    }
}
```

variantes/src/factorymethod/SMSNotification.java

```
package factorymethod;

public class SMSNotification extends Notification {
    @Override
    public void envoyer(String destinataire, String message) {
        String texte = message.length() > 160 ? message.substring(0, 157) + "..." : message;
        System.out.println("[SMS]   Vers " + destinataire + " : " + texte);
    }
}
```

variantes/src/factorymethod/PushNotification.java

```
package factorymethod;

public class PushNotification extends Notification {
    @Override
    public void envoyer(String destinataire, String message) {
        System.out.println("[PUSH]  Appareil " + destinataire + " -> \"" + message + "\"");
    }
}
```

variantes/src/factorymethod/NotificationFactory.java

```
package factorymethod;

/**
 * NotificationFactory : role FabriqueProduit.
 * La methode de fabrique creerNotification(canal) recoit le canal choisi
 * (EMAIL, SMS, PUSH) et decide quelle classe concrete instancier.
 */
public class NotificationFactory {
    public static final String EMAIL = "EMAIL";
    public static final String SMS = "SMS";
    public static final String PUSH = "PUSH";

    public Notification creerNotification(String canal) {
        Notification notification = null;
        switch (canal.toUpperCase()) {
            case EMAIL:
                notification = new EmailNotification();
                break;
            case SMS:
                notification = new SMSNotification();
                break;
            case PUSH:
                notification = new PushNotification();
                break;
            default:
```

```
                throw new IllegalArgumentException("Canal de notification inconnu : " +
canal);
        }
        return notification;
    }
}
```

variantes/src/factorymethod/Client.java

```
package factorymethod;


/**
 * Client : le canal est choisi a l'execution (ici via les arguments de la ligne de commande,
 * sinon les trois canaux sont testes). Le client ne connait que Notification et la fabrique.
 */
public class Client {
    public static void main(String[] args) {
        System.out.println(".....Variante 1 : Notifications (Factory
Method).....");
        NotificationFactory fabrique = new NotificationFactory();
        String[] canaux = args.length > 0 ? args : new String[] {"EMAIL", "SMS", "PUSH",
"FAX"};


        for (String canal : canaux) {
            try {
                Notification notification = fabrique.creerNotification(canal);
                notification.envoyer("etudiant@univ.cm", "Votre devoir INF4067 a ete recu.");
            } catch (IllegalArgumentException e) {
                System.out.println("Erreur : " + e.getMessage());
            }
        }
    }
}
```

5.4 Code source – fabrique abstraite

variantes/src/factoryabstraite/Notification.java

```
package factoryabstraite;


public abstract class Notification {
    public abstract void envoyer(String destinataire, String message);
}
```

variantes/src/factoryabstraite/EmailNotification.java

```
package factoryabstraite;


public class EmailNotification extends Notification {
    @Override
    public void envoyer(String destinataire, String message) {
        System.out.println("[EMAIL] A : " + destinataire + " : " + message);
    }
}
```

variantes/src/factoryabstraite/SMSNotification.java

```
package factoryabstraite;


public class SMSNotification extends Notification {
    @Override
    public void envoyer(String destinataire, String message) {
        System.out.println("[SMS]   Vers " + destinataire + " : " + message);
    }
}
```

variantes/src/factoryabstraite/PushNotification.java

```
package factoryabstraite;


public class PushNotification extends Notification {
    @Override
    public void envoyer(String destinataire, String message) {
        System.out.println("[PUSH]  Appareil " + destinataire + " : " + message);
    }
}
```

variantes/src/factoryabstraite/NotificationFactory.java
```
package factoryabstraite;

/**
 * Fabrique abstraite : declare la methode de fabrique creerNotification()
 * et implemente le traitement commun qui l'utilise (notifier).
 * Chaque sous-classe decide du type concret de notification.
 */
public abstract class NotificationFactory {
    public void notifier(String destinataire, String message) {
        Notification notification = creerNotification();
        notification.envoyer(destinataire, message);
    }

    protected abstract Notification creerNotification();
}
```

variantes/src/factoryabstraite/EmailNotificationFactory.java
```
package factoryabstraite;

/**
 * Fabrique concrete : sait creer une EmailNotification.
 */
public class EmailNotificationFactory extends NotificationFactory {
    @Override
    protected Notification creerNotification() {
        return new EmailNotification();
    }
}
```

variantes/src/factoryabstraite/SMSNotificationFactory.java
```
package factoryabstraite;

/**
 * Fabrique concrete : sait creer une SMSNotification.
 */
public class SMSNotificationFactory extends NotificationFactory {
    @Override
    protected Notification creerNotification() {
        return new SMSNotification();
    }
}
```

variantes/src/factoryabstraite/PushNotificationFactory.java
```
package factoryabstraite;

/**
 * Fabrique concrete : sait creer une PushNotification.
 */
public class PushNotificationFactory extends NotificationFactory {
    @Override
    protected Notification creerNotification() {
        return new PushNotification();
    }
}
```

variantes/src/factoryabstraite/Client.java
```
package factoryabstraite;

public class Client {
    public static void main(String[] args) {
        System.out.println(".....Variante 2 : Notifications (fabrique
abstraite).....");
        NotificationFactory[] fabriques = {
            new EmailNotificationFactory(),
            new SMSNotificationFactory(),
            new PushNotificationFactory()
        };
        for (NotificationFactory fabrique : fabriques) {
            fabrique.notifier("etudiant@univ.cm", "Votre devoir INF4067 a ete recu.");
        }
    }
}
```

5.5 Résultat de l'exécution

```
............Variante 1 : Notifications (Factory Method)............
[EMAIL] A : etudiant@univ.cm
        Objet : Notification
        Corps : Votre devoir INF4067 a ete recu.
[SMS]   Vers etudiant@univ.cm : Votre devoir INF4067 a ete recu.
[PUSH]  Appareil etudiant@univ.cm -> "Votre devoir INF4067 a ete recu."
Erreur : Canal de notification inconnu : FAX

............Variante 2 : Notifications (fabrique abstraite)............
[EMAIL] A : etudiant@univ.cm : Votre devoir INF4067 a ete recu.
[SMS]   Vers etudiant@univ.cm : Votre devoir INF4067 a ete recu.
[PUSH]  Appareil etudiant@univ.cm : Votre devoir INF4067 a ete recu.
```

6. Analyse et justification

Le système de notifications remplit les deux conditions d'utilisation du patron Factory données en cours. Le type d'objet n'est connu qu'à l'exécution, puisque le canal dépend de l'utilisateur. La création est centralisée dans la fabrique : les règles propres à chaque canal (format d'un e-mail, troncature d'un SMS à 160 caractères) sont dans les classes concrètes, et le choix de la classe est dans la fabrique.

Le client ne contient aucun `new EmailNotification()` : il manipule uniquement `Notification`. Remplacer un canal par un autre ou en ajouter un (WhatsApp, Telegram) n'impacte pas le client.

Forme	Avantage	Limite
Factory Method	Une seule fabrique, simple quand le choix dépend d'une valeur connue à l'exécution.	Le `switch` doit être modifié à chaque nouveau produit.
Fabrique abstraite	Ajouter un produit = ajouter une sous-classe de produit et une sous-classe de fabrique, sans modifier le code existant (principe ouvert/fermé).	Davantage de classes.

7. Dépôt GitHub

Le code source complet (exemple du cours et exemple modifié), les diagrammes DIA avec leurs exports PNG, le script de compilation et l'analyse sont disponibles à l'adresse :

https://github.com/LeonelAwouma/PatronDeConstruction_Factory_GOYE_AWOUMA_CHAINE_LEONEL

Dossier	Contenu
Cours/src/factorymethod	Exemple du cours, forme Factory Method
Cours/src/factoryabstraite	Exemple du cours, forme fabrique abstraite
Cours/uml	factory_method.dia, factory_abstraite.dia et PNG
variantes/src/factorymethod	Variante Notifications, forme Factory Method

Dossier	Contenu
variantes/src/factoryabstraite	Variante Notifications, forme fabrique abstraite
variantes/uml	Diagrammes DIA et PNG de la variante
documentation	analyse.md, rapport, scripts de génération
build.sh	Compile et exécute les quatre programmes (JDK 17)

Compilation et exécution :

```
javac -d out/cours $(find Cours/src -name '*.java')
java -cp out/cours factorymethod.Client
java -cp out/cours factoryabstraite.Client
javac -d out/variantes $(find variantes/src -name '*.java')
java -cp out/variantes factorymethod.Client SMS PUSH
java -cp out/variantes factoryabstraite.Client
```

8. Conclusion

Le patron Factory répond au problème du couplage fort introduit par l'opérateur `new`. Dans sa forme Factory Method, une fabrique unique choisit la classe à instancier d'après un paramètre reçu à l'exécution. Dans sa forme à fabrique abstraite, chaque fabrique concrète est responsable d'un produit, ce qui permet d'étendre le système sans modifier le code existant. L'exemple des notifications montre que le même modèle s'applique directement à un contexte réel, le client restant indépendant des classes concrètes.